

❖ Auto Scaling

❖ What is Auto Scaling?

- Auto Scaling is an AWS service that automatically adjusts the number of compute resources (like EC2 instances) in response to changing demand.
- It allows applications to **scale up** (add more resources) when demand **increases** and **scale down** (reduce resources) when **demand drops**.
- It leading to better performance **application availability**, **optimize cost**, and **maintain performance**.

✓ Key Objectives of Auto Scaling

1. *High Availability*
2. *Fault Tolerance*
3. *Elasticity*
4. *Cost Optimization*
5. *Demand Matching*
6. *Resource Optimization*

❖ Auto Scaling Group (ASG)

An **Auto Scaling Group** (ASG) is a **collection** of EC2 instances that are managed as a **single unit** for automatic scaling and high availability.

✓ Key Features of an ASG

Property	Description
Minimum Size	The minimum number of instances that must be running at all times.
Maximum Size	The maximum number of instances the group is allowed to scale up to.
Desired Capacity	The ideal number of instances to maintain under normal conditions.

❖ Dierence Between Launch Configuration and Launch Template

Feature	Launch Configuration	Launch Template ☒ (Recommended)
Modern Support	✗ Legacy (deprecated for new features)	☒ Fully supported and recommended
Multiple Versions	✗ Not supported	☒ Yes — supports versioning
Support for Newer EC2 Features	✗ Limited	☒ Full support (e.g., T2/T3 Unlimited, placement groups)
Separate Network Interfaces	✗ Not supported	☒ Can specify ENIs
Tagging Instances and Volumes	✗ Basic or no support	☒ Full support
Use with EC2 Fleet, Spot Fleet	✗ No	☒ Yes
Parameter Overriding	✗ No	☒ Supports parameter overriding at launch
Data Encryption at Launch	✗ No	☒ Yes
Mixed Instances Policy Support	✗ No	☒ Required for mixed instance types
Template Sharing	✗ No	☒ Can share across AWS accounts

❖ Scaling policies

Scaling policies are rules or configurations that automatically adjust the number of compute resources (such as EC2 instances) in response to changes in demand. These policies are used in Auto Scaling Groups (ASG) to maintain application performance and cost-efficiency.

✓ Key Types of Scaling Policies:

- **Dynamic Scaling:** *Automatically adjusts capacity based on real time metrics (usually from Amazon CloudWatch)*
 1. **Target Tracking Scaling Policy:**
 - Automatically adjusts capacity to maintain a specific metric target (e.g., keep average CPU utilization at 60%).
 - Simple and most commonly used.
 2. **Step Scaling Policy:**
 - Adds or removes instances in steps based on the magnitude of the metric breach.
 - Example: Add 1 instance if CPU > 60%, add 2 if > 80%.
 3. **Simple Scaling Policy (Legacy):**
 - Performs a single scaling activity based on a CloudWatch alarm.
 - Includes cooldown periods before subsequent scaling.
- **Scheduled Scaling:**
 - Scales resources up or down at specific times.
 - Useful for predictable traffic patterns (e.g., scale up weekdays 9 AM, scale down at night).

Example Scenario:

- **Metric:** CPU utilization
- **Target Tracking Policy:** Keep average CPU at 50%
- **Action:** If CPU > 50%, Auto Scaling will launch new instances; if < 50%, it will terminate instances.

❖ Benefits of Scaling Policies:

1. **Automatic resource adjustment** based on actual demand.
2. **Improves application performance** under load.
3. **Reduces cloud costs** by avoiding overprovisioning.
4. **Works with CloudWatch alarms**, metrics, and Auto Scaling Groups seamlessly

❖ Health check

Health Checks in Auto Scaling monitor the status of EC2 instances and ensure that only healthy instances are serving traffic. If an instance is marked as unhealthy, Auto Scaling automatically replaces it with a new one.

✓ 1. ELB (Elastic Load Balancer) Health Checks

Used to check the health of EC2 instances behind a Load Balancer.

✓ 2. EC2 Status Checks

Monitors the health of the instance itself.

✓ 3. Route 53 Health Checks

Used for **DNS failover and routing** decisions.

✓ 4. Container Health Checks (e.g., ECS, Kubernetes)

Defines how the orchestrator checks the container's health.

❖ Benefits of Health Checks

1. Keeps Applications Running Smoothly
2. Automatic Problem Detection
3. Supports Auto Healing
4. Improves User Experience
5. Enables High Availability
6. Used in Failover Systems

✓ Basic Working of Auto Scaling:

1. You Define a Group

- Called an **Auto Scaling Group (ASG)**, it contains a set of EC2 instances.
- You specify the **minimum, maximum, and desired number** of instances.

2. You Set Scaling Policies

- These are rules that tell when to **add** (scale out) or **remove** (scale in) instances.
- Example: Add 1 instance if CPU > 70% for 5 minutes.

[General Interview Questions]

◆ Q1. What is the purpose of using the stress command in Auto Scaling labs?

- Use the stress command in **Auto Scaling labs to generate high CPU or memory load intentionally**. This helps us test if Auto Scaling is working properly — like launching new instances when the load increases or terminating them when it drops."

Example:-

- `sudo yum install -y stress`
- `stress --cpu 2 --timeout 300`

This command loads 2 CPU cores for 5 minutes, helping trigger the scaling policy.

◆ Q2. How do you test a scale-in policy after stress testing?

- Stop the stress process. After cooldown and when CPU drops below the lower threshold, the ASG **will terminate extra EC2s**

◆ Q3. What are the components required for dynamic scaling setup?

- CloudWatch Alarm
- Auto Scaling Group
- Launch Template
- Scaling Policy (CPU > threshold)

*Note:- Threshold means A **threshold** is a **set limit** or **boundary value** that triggers an action when crossed.*

◆ Q4. What is Scheduled Scaling?

- **Scheduled Scaling** is a feature in Auto Scaling that allows you to **increase or decrease the number of instances at specific times**, based on a pre-defined schedule.

◆ Q5. Can Scheduled Scaling conflict with Dynamic Scaling?

- **Yes**. AWS always applies the most recent action, so overlap can cause undesired **scale-in** or **scale-out** if not configured properly.